Spring Boot

```java
package com.example.SpringWeb;


import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("student")
public class StudentController {


    // @Autowired
    StudentService service;

    public StudentController(StudentService service){
        this.service = service;
    }
    // student
    @GetMapping("/")
    public List<Student> Sayhello(){
        return service.getAllStudents();
    }

    // student/home

     @GetMapping("/home")
    public String SayHome(){
        return "Home";
    }

    // student/add
    @PostMapping("/add")
    public Student add(@RequestBody Student student){
        System.out.println(student.getId());
        System.out.println(student.getName());
        System.out.println(student.getMarks());
        return service.addStudent(student);
    }
```

```
    @GetMapping("/{id}")
    public Student getById(@PathVariable int id){

        return service.getStudentByid(id);

    }


    // /student/id

}
```

```
package com.example.SpringWeb;

import java.util.ArrayList;
import java.util.List;

import org.springframework.stereotype.Service;

@Service
public class StudentService {

    List<Student> students = new ArrayList<>();


    public StudentService(){
        students.add(new Student(1, "FIIT", 90));
        students.add(new Student(2, "REDMI", 92));

    }

    public List<Student> getAllStudents(){
        return students;
    }

    public Student addStudent(Student student){
        students.add(student);
        return student;
    }


    public Student getStudentByid(int id){

        for(Student student : students){
            if(student.getId() == id){
```

```
                return student;
            }
        }

        return null;
    }









}
```

```
package com.example.SpringWeb;

public class Student {

    private int id;
    private String name;
    private int marks;

    public Student(int id, String name, int marks) {
        this.id = id;
        this.name = name;
        this.marks = marks;
    }

    public int getId() {
        return id;
    }

    public String getName() {
        return name;
    }

    public int getMarks() {
        return marks;
    }

    public void setId(int id) {
        this.id = id;
    }
```

```
    public void setName(String name) {
        this.name = name;
    }

    public void setMarks(int marks) {
        this.marks = marks;
    }




    // ctrl + .  => dynamic ah generate constructor and setter and getter


}
```

📘 Theory

1. Controller Layer

- **@RestController** → Marks the class as a REST controller, returning JSON by default.
- **@RequestMapping("student")** → Base path for all endpoints.
- **@GetMapping**, **@PostMapping** → Maps HTTP methods to controller methods.
- **@PathVariable** → Extracts values from the URL.
- **@RequestBody** → Maps JSON request body to Java object.

2. Service Layer

- **@Service** → Marks the class as a service bean.
- Encapsulates business logic (like managing the student list).
- Keeps controllers clean by delegating logic.

4. Dependency Injection

- Constructor injection (StudentController(StudentService service)) → preferred approach.

- Ensures immutability and easier testing compared to field injection.

📝 Coding Round Questions

- Controller basics → What is the role of @RestController?
- Request mapping → How does @RequestMapping work with @GetMapping?
- Service layer → Why do we use @Service?
- Dependency injection → Why is constructor injection preferred over field injection?
- PathVariable vs RequestParam → Difference between @PathVariable

⚡ Machine Coding Round Challenge

Problem: Student CRUD API

Requirements:

1. Extend your current setup to support full CRUD:
    - GET /student/ → list all students.
    - GET /student/{id} → get student by ID.
    - POST /student/add → add a new student.
    - PUT /student/{id} → update student details.
    - DELETE /student/{id} → remove student.
2. Use StudentService to handle logic.
3. Return appropriate HTTP responses.